Jac on GPU: Matmul

Original Jac • checked compiler optimization • handwritten CUDA

How much performance can compiler optimization recover when every graph node stores one integer?

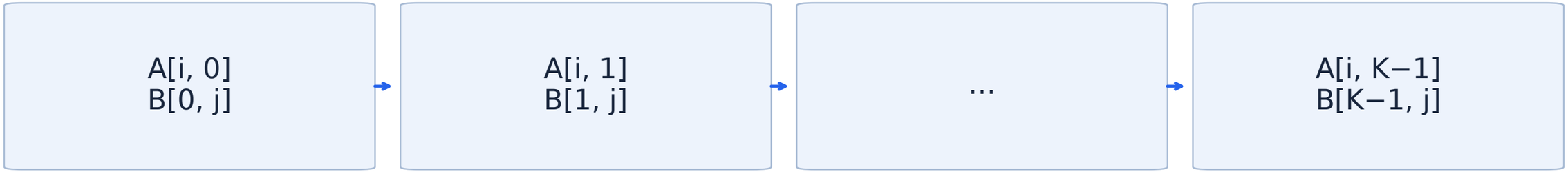
Separate a removable compiler cost from the remaining cost of general graph execution.

Measured on RTX 3090 • int64 arithmetic • full output validation

One output is a paired graph traversal

$C[i, j] = \Sigma q\ A[i, q] \times B[q, j]$

A cursor follows RowNext; B follows ColumnNext. Each paired arrival contributes one product to the output.



The production packer creates the input graph. A GPU lane performs a complete dot product; M × N lanes produce C.

Both Jac groups reuse identical packed arrays, seeds, lane order and launch geometry.

Three implementations, one arithmetic task

Jac layout is fixed to Predicted in the primary comparison.

Implementation	Execution strategy
Jac: no added optimization	Production LLVM → existing target codegen → PTX
Jac: with optimization	Same LLVM → checked LLVM O2 → target codegen → PTX
Handwritten CUDA	Fixed 16 × 16 shared-memory tiled int64 kernel

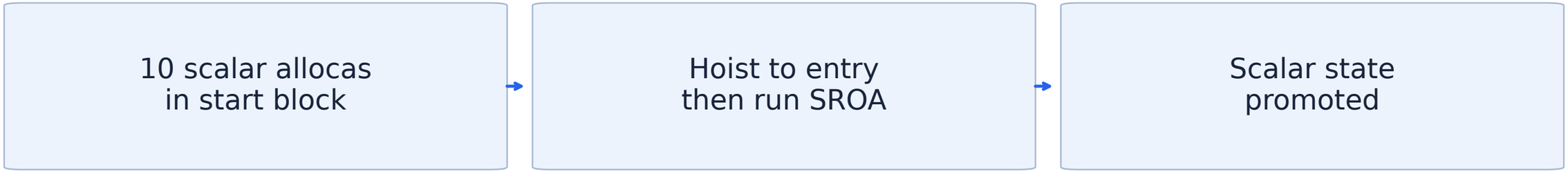
Checked O2 retains queues, type dispatch, overflow/bounds checks and status behavior. This compilation path is experimental.

The CUDA implementation uses dense arrays and cooperative tile reuse.

Optimization promotes cursor state to registers

A focused compiler transformation nearly reproduces the O2 result.

SROA alone leaves PTX unchanged. Offline ptxas reports zero register spills in both variants.



Loaded kernel / static PTX	Original	Checked O2
Local bytes per thread	80	0
Registers per thread	28	40
Static local loads / stores	20 / 20	0 / 0

Measurement: actual resident kernel duration

All primary cells come from one interleaved Nsight Systems run.

1. Use the same int64 A/B values and validate every output against NumPy.

2. Randomize implementation order each round; collect 240 kernel samples per cell after warmup

3. Report median GPU kernel end − start. Allocations, transfers, CPU work and graph scheduling gaps are outside this interval.

RTX 3090 • CUDA 12.9 • shared GPU: Ollama held approximately 22.9 GiB. Ratios use the interleaved run; absolute times reflect this environment.

Three-way performance comparison

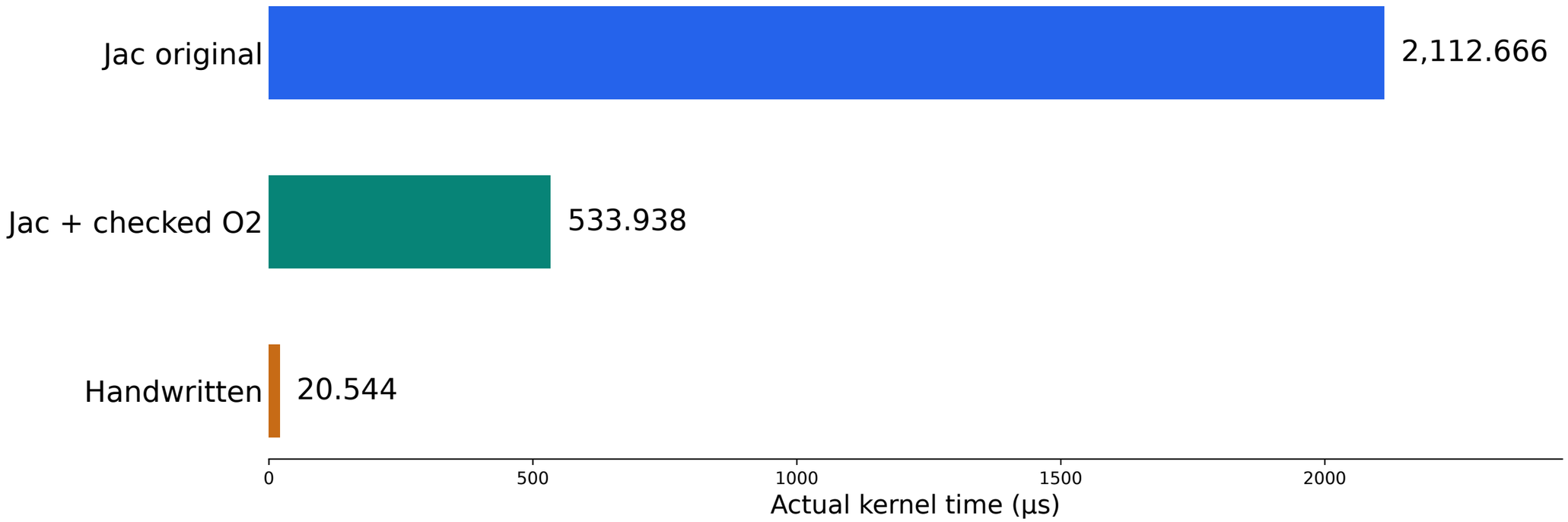
Median actual GPU kernel time in microseconds (µs); lower is faster.

M × K × N	Jac original	Jac + checked O2	Handwritten
3 × 5 × 2	8.384	7.264	1.568
17 × 31 × 19	52.736	45.216	2.176
64 × 128 × 64	183.745	155.872	4.704
128 × 128 × 128	228.113	164.577	4.832
256 × 256 × 256	2,112.666	533.938	20.544

256^3: 3.96× faster after O2; optimized Jac / handwritten = 25.99×.

At 256³, O2 recovers a substantial compiler cost

The same fixed handwritten tiled kernel is used for every matrix shape.



2.113 ms → 0.534 ms → 0.0205 ms

What the remaining gap tells us

Compiler quality and graph execution strategy both deserve attention.

Established: scalar-state promotion removes local-memory accesses and nearly reproduces O2’s speedup.

After O2: generic FIFO traversal, CSR lookups, dispatch and checks still execute. Restricted chain controls run faster on regular layouts.

Those controls change the execution contract and compiler. The remaining 26× gap is a whole-implementation comparison.

Cache, coalescing and occupancy attribution remains open: this study has timing traces and codegen evidence; hardware counters were unavailable.

Evidence and reproducibility

Source commit e476272e4 • scripts/gpu_walker/handwritten_matmul/

Evidence	Validation
Primary profiler run	11,560 launches matched; 9,600 measured
Supplementary ablation run	43,350 launches matched; 36,000 measured
Direct GPU correctness	13 fixtures × 3 checked variants = 39 passes
Named-cursor CPU / JIT suite	15 tests passed

REPORT.md • primary/comparison.csv • codegen/ABLATIONS.md

reproduce_codegen.sh rebuilds variants, profiles execution and validates results. This deck checks source hashes and primary table values when built.